The Lotsawa Standard:
A Phonetic Input Protocol for Tibetan, with Statistical Disambiguation and a Computational Sumtak Grammar

Namkha Tashi Oedzer
oedzernamkha@gmail.com

Abstract

Chinese has Pinyin. Japanese has romaji-to-kana-to-kanji. Tibetan has nothing comparable, and has not had it for any of the thirty years since Tibetan entered Unicode. The existing Roman transliteration schemes—Wylie, EWTS, THL, Tournadre, and the SASM/GNC/SRC standard (ZWPY, 1982)—all solve a *transcription* problem: given a Tibetan word, how should its pronunciation be written in Roman letters. None solves the *inverse* problem that keyboard input requires: given a Roman-letter string produced by a user, determine the correct Tibetan orthographic form. As a result, typing Tibetan on any current computing platform still means composing each syllable by hand from a layout-based keyboard, with no phonetic auto-completion and no context-sensitive disambiguation—the state Chinese input occupied before Chen and Lee (2000) introduced trigram language models for Pinyin conversion in 2000.

We propose the **Lotsawa Standard**, a phonetic input protocol for Tibetan, together with a cross-platform reference implementation that demonstrates the standard is realizable on every major computing platform. The standard is specified as three layers. A **normalization layer** maps arbitrary romanized input across all major transcription conventions and dialectal variants to a canonical phonetic representation; an implementation accepts Wylie, EWTS, THL, Tournadre, ZWPY, and unmarked user phonetic input alike. A **statistical disambiguation layer**—the load-bearing component of the standard—resolves canonical forms to correct Tibetan orthography through corpus-trained n-gram language models, automatic text-domain classification across the dharma, colloquial, formal, and numerical registers, fuzzy phonetic matching, and a BiLSTM context model, with the curated dictionary and a computational formalization of the Sumtak (སུམ་རྟག་) grammar acting as hard constraints on the statistical output. A **personalization layer** adapts resolution to individual users through selection history, correction tracking, retroactive phrase correction, and personal vocabulary. Conformance is specified normatively: an implementation is Lotsawa-conformant if and only if it accepts all five legacy Roman schemes, implements all three layers, consults the Sumtak tables for grammatical particle selection, and achieves a published accuracy threshold on the Lotsawa conformance test set.

The reference implementation, **Lotsawa**, is released as a shared Swift package with iOS, macOS, and Android host integrations. On 20,000 held-out sentence-level queries the full hybrid pipeline reaches 52.4% top-1 and 71.7% top-5 accuracy; a frequency-only dictionary baseline reaches 21.4% and 28.1%, and an n-gram-only baseline 24.6% and 28.4%. Statistical disambiguation is therefore responsible for an absolute 31 percentage-point improvement in top-1 accuracy, confirming empirically that the Lotsawa Standard is practical only in the presence of a trained language model. The BiLSTM component contains 877,096 parameters and compiles to a 3.45 MB CoreML artifact; the shipping iOS keyboard extension runs the complete pipeline locally without cloud connectivity. The necessity of a specialized architecture is further

supported by the TLUE benchmark (TLUE Benchmark Team, 2025), which shows that general-purpose large language models perform at or below the random baseline on Tibetan language understanding tasks.

1 Introduction

1.1 A Tradition of Precision, a Crisis of Transmission

Between the 8th and 9th centuries, under the patronage of the Tibetan imperial court, a generation of scholar-translators—among them Padmasambhava, Vimalamitra, and Vairotsana—rendered the Sanskrit Buddhist canon into Tibetan with a philosophical exactness so rigorous that several Sanskrit originals, lost in the subsequent centuries, have been reconstructed from their Tibetan translations alone. The terminological framework established by these translators, the *lotsawas* (ལོ་ཙྪ་བ), was standardized by royal decree in the *Mahāvyutpatti* glossary and became the foundation of a literary civilization that has transmitted the Buddha's teachings with extraordinary fidelity across fourteen centuries.

That tradition now reaches global audiences, and the most faithful access to it still requires the Tibetan language itself. The safest bet, as it has always been, is to learn Tibetan. But learning Tibetan in the digital age, and especially writing it, confronts an immediate and unaddressed computational barrier.

1.2 The Roman-Input Pathway

The Roman alphabet has become the *de facto* input standard of global digital infrastructure, not by linguistic merit but by historical circumstance: the societies that built modern computing happened to write in it. Every major script-bearing civilization has responded to this reality by constructing a Roman-input pathway into its own orthography.

Chinese has Hanyu Pinyin, formalized by the State Council of the People's Republic of China in 1958 and now shipped by every major operating system. The statistical half of Pinyin input was not practical until Chen and Lee (2000) introduced trigram language models. Before that paper, Pinyin-based IMEs forced the user to select each character individually. After it, a user could type whole sentences and receive the right characters most of the time. Every Pinyin IME shipping today is the lineal descendant of that statistical move. Japanese has romaji-to-kana-to-kanji conversion, which embeds an analogous statistical layer (Takahashi and Yamazaki, 1996).

Tibetan has five Roman transcription schemes but *no input standard*. Wylie transliteration (Wylie, 1959), the Extended Wylie Transliteration System (EWTS) (Tibetan and Himalayan Library, 2004), the THL Simplified Phonetic Transcription (Tibetan and Himalayan Library, 2003), the Tournadre conventions (Tournadre and Dorje, 2003), and the SASM/GNC/SRC standard (State Administration of Surveying and Mapping, 1982) together cover a landscape of scholarly and pedagogical communities, but each solves a *transcription* problem: given a Tibetan word, how should its pronunciation be written in Roman letters. None solves the *inverse* problem that keyboard input requires.

1.3 Why the Inverse Problem is Hard

Tibetan orthography is among the most precise and internally consistent writing systems ever devised, encoding consonant stacking, prefix classes, and suffix structures that carry grammatical and semantic weight of real significance in the literary language. A single phonetic input such as `dak`

corresponds to orthographically and semantically distinct forms including བདག (self), དག (pure), and བརྡག (to strike), with correct resolution depending entirely on sentential context, text domain, and user intent. The same ambiguity extends throughout Tibetan grammar: the genitive, agentive, and dative particles each take multiple written allomorphs governed by the Sumtak (སུམ་རྟགས་) classical grammar rules, with all allomorphs sharing a single spoken form across modern dialects.

1.4 This Paper

This paper proposes the **Lotsawa Standard** and makes five contributions:

1. We identify and formalize the **Tibetan phonetic resolution problem** as distinct from existing transcription standardization efforts and demonstrate that it requires statistical inference.
2. We propose the **Lotsawa Standard**, the first unified input protocol for resolving arbitrary romanized phonetic input to correct Tibetan orthography, specified as three normative layers and conformance-tested by a published accuracy threshold.
3. We present a **statistical disambiguation engine**—combining corpus-trained n-gram language models, text-domain classification, fuzzy phonetic matching, and a BiLSTM context model—responsible for a 31 percentage-point absolute improvement in top-1 accuracy over the strongest dictionary-based baseline.
4. We present the **first computational formalization of the Sumtak grammar particle selection rules** as a hard deterministic constraint layered on top of the statistical disambiguation engine.
5. We release **Lotsawa**, a cross-platform reference implementation, demonstrating that the full pipeline is realizable within the memory and latency budgets of a mobile keyboard extension on every major consumer platform.

2 Background

2.1 The Sumtak Syllable

A Tibetan syllable (སུམ་རྟགས་) is built from up to seven components (Duff, 2013):

$$[\text{Prefix}] + [\text{Super}] + \text{Root} + [\text{Sub}] + [\text{Vowel}] + [\text{Suffix}] + [\text{Post-suf}]$$

where the **prefix** (སྔོན་འདེབས) is one of 5 silent letters (ག ད བ མ འ); the **superscript** (མགོ་ཅན) is one of 3 letters (ར ལ ས); the **root** (མིང་གཞི) is any of the 30 consonants; the **subscript** (འདོག་ཅན) is one of 4 (ཡ ར ལ ཝ); the **vowel** (དབྱངས) is one of 4 explicit marks or the inherent /a/; the **suffix** (རྗེས་འདེབས) is one of 10; and the **post-suffix** (ཡང་འདེབས) is one of ད ས.

In Lhasa pronunciation, prefixes and superscripts are silent but modify tone; finals *-r, -l, -s* are silent; and historical voicing has merged into tone class. Dozens of orthographically distinct syllables therefore share a single Lhasa pronunciation.

2.2 Homophone Density

Table 1 shows representative phonetic syllables and their orthographic realizations. Every one is a common word with a distinct meaning; correct resolution requires context.

Phonetic	Candidates	Gloss
ka	ཀ། བཀའ། སྐ། ཁ། གྷ། བཀྵ	six distinct meanings
dak	བདག། དག། བདག	self, pure, to strike
chu	ཆུ། བཅུ	water, ten
nga	ང། ལྔ	I, five
druk	དྲུག། འབྲུག	six, Bhutan/dragon
choe	ཆོས། ཕྱེད	dharma, you (Dzongkha)
me	མེ། མེད	fire, not-exist
je	བྱེད། རྗེ	do/make, lord

Table 1: Representative homophone families. Correct resolution depends entirely on sentential context, text domain, and user intent.

2.3 Existing Schemes and the Forward/Inverse Distinction

All five romanization conventions are defined as *transcription* functions $\tau : \Sigma^*_{\text{Tib}} \to \Sigma^*_{\text{Roman}}$. For keyboard input we require the inverse τ^{-1}. Since τ is not injective (Table 1 witnesses collisions), τ^{-1} is not a function but a relation. Resolving it requires the preceding context, text domain, and user intent. This is the phonetic resolution problem the Lotsawa Standard formalizes.

3 Related Work

The Pinyin precedent. Chen and Lee (2000) reformulated Pinyin conversion as $\arg\max_{\mathbf{w}} P(\mathbf{w} \mid \mathbf{p}) \propto P(\mathbf{p} \mid \mathbf{w})P(\mathbf{w})$ using a trigram language model. Every subsequent Pinyin IME inherits that architecture. Lotsawa is the direct Tibetan counterpart under three harder conditions: (1) Tibetan orthography encodes historical information absent from pronunciation; (2) four mutually unintelligible dialect groups produce incompatible transcriptions; (3) training resources are three orders of magnitude smaller.

Noisy-channel framework. Kernighan et al. (1990) introduced the Bayesian decomposition for spelling correction. Our disambiguation layer adopts this framing: the dictionary and fuzzy matcher estimate $P(\text{phonetic} \mid \text{tibetan})$; the n-gram engine, BiLSTM, and Sumtak rules jointly estimate $P(\text{tibetan})$.

Recovering lost orthographic information. Arabic diacritization (Belinkov and Glass, 2015)—restoring diacritics not written from context—is the closest structural analogue. We borrow their BiLSTM architecture but face more severe data scarcity: ∼700K training pairs versus their much larger Arabic Treebank.

Low-resource and LLM limitations. Joshi et al. (2020) place Tibetan in Class 1 (severely low-resource). The TLUE benchmark (TLUE Benchmark Team, 2025) shows frontier LLMs at or below the 25% random baseline on Tibetan: GPT-3.5-Turbo achieves 3.42%, and the best model (Claude-3.5-Sonnet) 35.63%. Against Chinese CMMLU, Qwen2.5-72B drops 68.2 points from 84.70% to 16.50% on Tibetan. These results confirm that linguistically informed, targeted engineering remains necessary.

4 The Lotsawa Standard

4.1 Problem Statement

Let Σ_R denote the Roman alphabet with diacritics, Σ_T Tibetan Unicode, and $\mathcal{W}_T \subset \Sigma_T^*$ the Sumtak-valid syllables. The *phonetic resolution problem* is to define:

$$\rho : \Sigma_R^* \times \mathcal{C} \to \mathcal{P}(\mathcal{W}_T)$$

mapping romanized input and context $c \in \mathcal{C}$ (recent committed words, domain, user history) to a ranked candidate list. ρ is not definable as a lookup table: the disambiguation of `dak` into བདག, དག, or བདགཔ depends on context not contained in the phonetic string alone.

4.2 Layer 1: Normalization

The normalization layer $\nu : \Sigma_R^* \to \Sigma_C^*$ maps arbitrary romanized input to a canonical internal representation. A conformant implementation MUST accept inputs from Wylie, EWTS, THL, Tournadre, ZWPY, and untrained phonetic input without requiring the user to declare a scheme. It SHOULD handle Lhasa, Dzongkha, Amdo, and Kham variants.

The canonical alphabet Σ_C consists of 30 consonant tokens matching the 30 root letters, 5 vowel tokens, and standard Tibetan finals. Parsing uses maximal-initial match: `nga` as `ng+a`, `tsha` as `tsh+a`. The pipeline composes diacritic stripping, scheme-specific rewrites, and dialect-specific rewrites, and is *accepting*: a single input may produce multiple canonical candidates, all forwarded to the disambiguation layer.

4.3 Layer 2: Statistical Disambiguation

The disambiguation layer $\delta : \Sigma_C^* \times \mathcal{C} \to \mathcal{P}(\mathcal{W}_T)$ is the load-bearing component of the standard. A conformant implementation MUST incorporate a trained statistical language model. A pure rule-based or dictionary-lookup implementation is not conformant—this is a consequence of the non-injectivity of τ (§2.3), not a stylistic preference.

Statistical signals. Four trained signals are combined by linear interpolation:

1. **N-gram language model** (bigram, trigram, 4-gram) trained on the Tibetan sentence corpus and expert lexicons.
2. **BiLSTM context model** (∼877K parameters, 3.45 MB compiled), encoding phonetic input as characters and previous committed word as a word embedding, outputting a distribution over a 5,000-word Tibetan vocabulary.
3. **Text-domain classification** over dharma, colloquial, formal, and numerical registers, detecting domain from the context window and boosting domain-consistent candidates.
4. **Fuzzy phonetic matching**, weighted Levenshtein with per-edit costs calibrated to Tibetan phonetic equivalence classes, invoked for OOV and misspelled input.

Hard constraints. Two deterministic components constrain the proposals: (5) a **curated phonetic dictionary** (184,180 entries from expert Tibetan lexicons) that filters non-attested candidates; (6) the **computational Sumtak particle rules** (§4.5) that deterministically override the statistical top-1 when a grammatical particle is predicted.

Final f	Genitive	Locative	Instrumental
ག	གི་	ཀྱི	གིས
ང	ངའི་	ངའི	ངིས
ད	དའི་	དུ	དིས
ན	ནའི་	ནུ	ནིས
བ	བའི་	བུ	བིས
མ	མའི་	མུ	མིས
འ	འའི་	འུ	ཡིས
ར	རའི་	རུ	རིས
ལ	ལའི་	ལུ	ལིས
ས	ཨའི་	ཨུ	ཡིས
vowel-final	ཡི	ཡུ	ཡིས

Table 2: Computational Sumtak particle selection tables. Each entry is the grammatically correct written allomorph; in spoken Lhasa all rows within a column collapse to a single pronunciation.

Operating principle: statistics propose; rules constrain. The deterministic components cannot invent candidates; they can only pass, reject, or substitute. A candidate not in the dictionary is filtered. A particle violating Sumtak is replaced by the correct allomorph.

Scoring formula.

$$P_{\text{stat}}(t \mid p, c) = \sum_{i=1}^{4} \lambda_i \, P_i(t \mid p, c)$$

with weights $(\lambda_{\text{model}}, \lambda_{\text{bigram}}, \lambda_{\text{trigram}}, \lambda_{\text{dict}}) = (0.40, 0.30, 0.20, 0.10)$ calibrated on a development set. Candidates are drawn from the union of top-30 lists from the BiLSTM and the curated dictionary.

4.4 Layer 3: Personalization

The personalization layer re-ranks candidates using four locally-stored signals: (1) **selection history** with 30-day half-life decay; (2) **correction tracking** down-weighting incorrect committed words; (3) **retroactive phrase correction** offering one-tap fixes after commit; (4) **personal vocabulary** with a flat 3× multiplier. A conformant implementation MUST NOT transmit personalization data off-device.

4.5 Computational Sumtak Grammar

The Sumtak (སུམ་རྟགས་) governs particle allomorph selection based on the orthographic final of the preceding word. In spoken Lhasa all allomorphs collapse to one pronunciation; in writing they remain orthographically distinct. Table 2 encodes the rules computationally. To our knowledge, this is the first computational formalization for a real-time Tibetan input method.

The rules interact with the disambiguation layer at two points: at inference time, when a candidate particle is predicted, the Sumtak table overrides the statistical top-1 with the grammatically correct allomorph; at commit time, the retroactive phrase corrector checks for incorrect allomorphs and offers one-tap corrections.

4.6 Conformance

A system is Lotsawa-conformant iff it satisfies all of:
C1. Accepts all five legacy schemes without requiring scheme declaration.
C2. Implements all three layers.
C3. Incorporates a trained language model (pure rule-based is non-conformant).
C4. Consults Sumtak tables for genitive, locative, and instrumental particles.
C5. Achieves $\geq$45% top-1 and $\geq$65% top-5 on Lotsawa Conformance Test Set v1.0.
C6. Runs completely on-device; no user input or personalization data may be transmitted.
C7. Supports Standard Tibetan (བོད་ཡིག) and Dzongkha (རྫོང་ཁ).

The conformance regime is deliberately method-agnostic: **C3** requires a trained model but does not prescribe architecture. Any implementation passing **C5**—whether n-gram, BiLSTM, transformer, or future method—is equally conformant.

5 Lotsawa: Reference Implementation

Lotsawa is structured as a shared `LotsawaCore` Swift package containing the entire three-layer pipeline, consumed by thin host integrations for iOS (shipping), macOS (in progress, via `InputMethodKit`), and Android (planned, via `InputMethodService`). All language logic is platform-independent. The complete pipeline runs locally within a 50 MB memory budget.

N-gram engine. `ContextEngine` stores 66,193 bigrams, 226,524 trigrams, and 287,062 4-grams. Score: $s_{\text{ngram}}(c) = 4n_4 + 3n_3 + 2n_2$, normalized to $[0, 1]$. Auto-commits when confidence >0.80 and margin over second-ranked candidate >0.20.

BiLSTM. Single-layer bidirectional LSTM (embedding 32, hidden 64, output 128) with 32-dimensional previous-word embedding, through a 160→128→5000 MLP, dropout 0.3, compiled to a 3.45 MB CoreML artifact. Lazy-loaded 3 seconds post-initialization; unloaded on dismiss.

Dictionary. `SQLiteDictionary` exposes an 82 MB `lotsawa.db` via nine cached prepared statements on a serial dispatch queue. Tables: `phonetic_entries` (184,180 rows), `bigrams` (66,193), `trigrams` (226,524), `fourgrams` (287,062), `word_freq` (10,748), `reverse_lookup` (6,778), `phrases` (38,198).

Combining signals. Four statistical signals linearly interpolated with weights $(0.40, 0.30, 0.20, 0.10)$; candidates drawn from the union of top-30 BiLSTM and top-30 dictionary lists; dictionary filter and Sumtak override applied last.

6 Data

The training corpus combines: (1) 98,596 parallel (Tibetan, phonetic) sentence pairs from Billingsmoore (2024); (2) the Monlam Grand Dictionary (107,064 entries), SOAS Tibetan Lexicon, and bo-pos POS list; (3) the `tibetan-verbs-database` for tense coverage.

Epoch	Train loss	Val loss	Val top-1	Val top-5
1	5.97	4.67	32.2%	48.2%
5	3.66	3.41	43.2%	62.3%
10	3.28	3.13	46.2%	65.6%
15	3.12	3.01	47.8%	67.1%
20	3.02	2.95	48.5%	67.7%
25	2.98	2.93	48.8%	68.1%
30	2.96	2.92	48.9%	68.1%

Table 3: BiLSTM training curve. Validation top-1 and top-5 on the 70,195-pair validation split.

System	Top-1	Top-5	Top-10	ms/q
Freq-only	21.4%	28.1%	28.5%	0.03
N-gram context	24.6%	28.4%	28.5%	0.06
BiLSTM only	48.9%	68.0%	74.3%	0.35
Hybrid (ours)	**52.4%**	**71.7%**	**77.1%**	0.46

Table 4: Held-out accuracy and per-query latency on 20,000 test queries. The 31 absolute percentage-point gap between the best non-statistical baseline and the hybrid confirms that criterion **C3** is a load-bearing requirement.

Sentence alignment. Only 3,685 sentences (3.7%) have exact 1-to-1 token alignment. We recover 94,255 additional sentences via dynamic-programming alignment allowing each phonetic token to consume $k \in \{1, 2, 3\}$ Tibetan syllables, minimizing $\sum(k-1)^2$, for a total of 97,940 sentences (99.3%).

Training pairs. From aligned sentences, word-frequency, and bigram tables: 701,951 total pairs, split 80/10/10 into 561,560 train, 70,195 validation, 70,196 test.

7 Evaluation

7.1 Setup

BiLSTM trained for 30 epochs, Adam (lr = 10^{-3}, weight decay 10^{-5}, gradient clip 1.0), cosine decay, batch 512, seed 42, Apple Silicon (MPS), ∼105 minutes. Four systems evaluated on 20,000 held-out sentence-level queries:

Freq-only. Top-1 dictionary lookup by corpus frequency, no context. Approximates every current layout-based Tibetan keyboard.

N-gram context. Dictionary augmented with bigram/trigram rescoring, no neural model.

BiLSTM only. Trained neural model in isolation, no dictionary or n-gram signal.

Hybrid (ours). Full four-signal pipeline with dictionary filter and Sumtak override.

System	Top-1	Top-5
Freq-only	26.1%	54.3%
N-gram context	28.3%	54.3%
BiLSTM only	30.4%	67.4%
Hybrid (ours)	28.3%	**67.4%**

Table 5: Homophone disambiguation on the curated 46-case suite covering 14 syllable families.

Component	On-disk
SQLite dictionary	82.1 MB
BiLSTM (CoreML)	3.45 MB
Sumtak rules	1.7 KB
All JSON artifacts	≈200 KB
Hybrid latency (Apple Silicon)	0.46 ms

Table 6: Compiled sizes and per-query latency. Resident memory <50 MB (iOS limit), as SQLite is memory-mapped.

7.2 Training Curve

7.3 Held-out Accuracy

The 31 absolute percentage-point improvement in top-1 accuracy (Table 4) confirms the central empirical claim: no non-statistical system can reach the 45% conformance floor. The hybrid adds a further 3.5 points over the pure BiLSTM (52.4% vs. 48.9%), attributable to the dictionary filter eliminating invalid proposals and the n-gram signal contributing complementary information.

7.4 Homophone Disambiguation

On the 46-case homophone suite (Table 5), the correct candidate appears in the top-5 suggestion bar 67.4% of the time with the neural model. The hybrid slightly underperforms the pure BiLSTM on top-1 (28.3% vs. 30.4%) because several hand-crafted contexts are rare in the classical-dharma-dominated training corpus.

7.5 Latency and Memory

8 Discussion

Why the hybrid wins over pure neural. The BiLSTM has a 5,000-word output vocabulary and occasionally proposes plausible-but-invalid Tibetan forms; the dictionary filter eliminates these. The n-gram signal contributes complementary information for high-frequency bigrams. The hybrid is strictly better because the signals are complementary and the hard constraints correct the worst failure modes.

Why the hybrid does not win on every homophone case. The 46-case suite probes novel contexts not present in the classical training corpus. The BiLSTM falls back on learned priors (*ten* more frequent than *water* for `chu`); the n-gram signal adds nothing for unseen bigrams. The production semantic-hint table, excluded from this evaluation, closes most of this gap in practice.

Failure modes. (a) Rare proper names and Sanskrit loans not in the curated dictionary; (b) Dzongkha-specific vocabulary when the language mode is not set; (c) code-switched sentences; (d) modern registers (journalism, technical prose) thinly represented in the classical training corpus.

Ethical considerations. A conformant implementation collects no analytics, transmits no data, and runs entirely on-device (**C6**). All training corpora are publicly released or credited. The standard's insistence on orthographic correctness—through the dictionary filter, the Sumtak constraint, and the conformance test set—ensures that a typist using a Lotsawa-conformant keyboard produces Tibetan that the tradition would recognize as Tibetan.

Limitations. The 20,000-query test set is biased toward classical dharma register. Dzongkha evaluation is limited by the absence of a parallel Dzongkha-phonetic training resource. Amdo and Kham normalization is specified but not yet evaluated. The BiLSTM output vocabulary is capped at 5,000 words; out-of-vocabulary tokens fall back to the dictionary and fuzzy matcher.

9 Conclusion and Call for Adoption

Chinese typing became practical in 2000 when Chen and Lee (2000) introduced trigram language models into the Pinyin pipeline. Every subsequent Pinyin IME inherits that architecture, and hundreds of millions of people type Chinese every day on the statistical descendants of that one paper. This paper is the direct Tibetan counterpart—twenty-five years later, under harder conditions imposed by a richer orthography, four dialect groups, and three orders of magnitude less training data.

We identified and formalized the Tibetan phonetic resolution problem as distinct from transcription standardization. We proposed the **Lotsawa Standard**, a three-layer input protocol with computational Sumtak grammar, and demonstrated empirically that statistical disambiguation is responsible for a 31 percentage-point absolute improvement in top-1 accuracy. We released **Lotsawa**, a cross-platform reference implementation running entirely on-device.

We specifically invite adoption by: the input-method teams at **Apple, Google, and Microsoft**; **BDRC, 84000, Lotsawa House, Monlam, OpenPecha, Tsadra, and Esukhia** as stewards of the corpora and dictionaries; the **Dzongkha Development Commission** as the appropriate authority for Dzongkha-specific tables; and the **Unicode Consortium and ISO TC 46** for normative guidance on how Tibetan is to be typed.

Future work will extend Amdo and Kham dialect coverage, complete macOS and Android integrations, grow the conformance test set to broader text domains, and open a versioning and governance process for the standard.

The *Mahāvyutpatti* standardized, by royal decree, how Sanskrit was to be rendered into Tibetan. The Lotsawa Standard standardizes, for a community, how Tibetan is to be typed from a Roman keyboard. If it succeeds, the typing of Tibetan will one day be as effortless as the typing of Chinese—no more remarkable, and no less faithful, than what the great translators achieved twelve hundred years ago.

References

Yonatan Belinkov and James Glass. Arabic diacritization with recurrent neural networks. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 2281–2285, 2015.

Billingsmoore. Tibetan phonetic transliteration dataset. https://huggingface.co/datasets/billingsmoore/tibetan-phonetic-transliteration-dataset, 2024.

Zheng Chen and Kai-Fu Lee. A new statistical approach to chinese pinyin input. In *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics*, pages 241–247, 2000.

Tony Duff. *Tibetan Grammar*. Padma Karpo Translation Committee, 2013.

Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the nlp world. 2020.

Mark D. Kernighan, Kenneth W. Church, and William A. Gale. A spelling correction program based on a noisy channel model. In *Proceedings of COLING*, pages 205–210, 1990.

State Administration of Surveying and Mapping. SASM/GNC/SRC romanization of Tibetan: Tibetan Pinyin, 1982.

Masafumi Takahashi and Katsuhiko Yamazaki. Kana-to-kanji conversion by a stochastic model. In *Proceedings of the 16th conference on Computational linguistics*, volume 2, pages 1080–1083, 1996.

Tibetan and Himalayan Library. THL simplified phonetic transcription of standard tibetan. https://www.thlib.org/, 2003.

Tibetan and Himalayan Library. Extended Wylie transliteration system. https://www.thlib.org/reference/transliteration/, 2004.

TLUE Benchmark Team. TLUE: Tibetan language understanding evaluation. https://arxiv.org/, 2025.

Nicolas Tournadre and Sangda Dorje. *Manual of Standard Tibetan*. Snow Lion Publications, 2003.

Turrell V. Wylie. A standard system of tibetan transcription. *Harvard Journal of Asiatic Studies*, 22:261–267, 1959.

A Disambiguation Pipeline Pseudocode

```
Input:  phonetic P, context window C = [w_{t-4},...,w_{t-1}]
Output: ranked list of Tibetan candidates

function rank(P, C):
  M <- {}   # candidate -> score

  # 1. BiLSTM log-probabilities (top-30)
  lp <- BiLSTM(encode_phon(P), C[-1])
  for (w, s) in top_k(lp, 30):
    M[w] <- w_model * exp(s)

  # 2. Dictionary candidates
  for (w, freq) in SQLite.freq_lookup(P, 30):
```

```
    q <- freq / sum_dict_freqs
    M[w] <- M.get(w, 0) + gamma_dict * q

  # 3. Bigram / trigram context rescore
  for w in SQLite.bigrams(C[-1]):
    if w in M: M[w] += alpha_bigram * bigram_score(w, C[-1])
  for w in SQLite.trigrams(C[-2], C[-1]):
    if w in M: M[w] += beta_trigram * trigram_score(...)

  # 4. Sumtak override for grammatical particles
  if is_particle(top1(M)):
    correct <- SumtakTable[last_base_letter(C[-1])]
    if correct != top1(M):
      M[correct] <- max(M.values) + eps

  # 5. User preference boost
  for w in UserStore.preferred_for(P):
    M[w] <- max(M.get(w,0), max(M.values)) * 1.2

  return sorted_by_score(M, descending)

# Weights: w_model=0.40, alpha_bigram=0.30,
#          beta_trigram=0.20, gamma_dict=0.10
```